

Bài 21: Modules trong Java

Module là cơ chế đóng gói và quản lý các package được giới thiệu từ Java 9. Module giúp kiểm soát package nào được public, module nào được phép sử dụng dependency nào và giảm sự phụ thuộc không cần thiết giữa các phần của ứng dụng.

Java Module System còn được gọi là JPMS (Java Platform Module System).

1. Vấn đề trước khi có Module

Trước Java 9, ứng dụng Java chủ yếu được tổ chức bằng package và JAR.

Ví dụ:

```
application.jar
  com.example.service
  com.example.repository
  com.example.model
```

Cách tổ chức này có một số hạn chế:

- Mọi public class trong JAR đều có thể được module khác sử dụng.
- Khó kiểm soát dependency giữa các package.
- Có thể xảy ra tình trạng dependency bị thiếu hoặc xung đột phiên bản.
- Package nội bộ cũng có thể bị truy cập nếu class được khai báo public.
- Classpath không kiểm tra dependency một cách chặt chẽ.

Module giúp giải quyết các vấn đề này bằng cách khai báo rõ ràng ranh giới và dependency.

2. Module là gì?

Module là một nhóm package có:

- Tên duy nhất.
- Danh sách module phụ thuộc.
- Danh sách package được phép public ra bên ngoài.
- Có thể cung cấp hoặc sử dụng service.

Module được khai báo trong file đặc biệt:

```
module-info.java
```

Một module có thể chứa nhiều package:

```
my.app
├─ module-info.java
├─ com.example.app
│   └─ Main.java
│   └─ User.java
```

3. Cấu trúc module-info.java

Cấu trúc cơ bản:

```
module com.example.app {
    requires java.sql;
    exports com.example.app.api;
}
```

Ý nghĩa:

- module com.example.app: khai báo tên module.
- requires java.sql: module này sử dụng module java.sql.
- exports com.example.app.api: public package được phép cho module khác sử dụng.

4. Tạo module đơn giản

Cấu trúc thư mục:

```
src
├─ com.example.app
│   └─ module-info.java
│   └─ com
│       └─ example
│           └─ app
│               └─ Main.java
```

File module-info.java:

```
module com.example.app {
}
```

File Main.java:

```
package com.example.app;

public class Main {
```

```
public static void main(String[] args) {  
    System.out.println("Hello Module");  
}  
}
```

Biên dịch:

```
javac -d out --module-source-path src -m com.example.app
```

Chạy:

```
java --module-path out -m com.example.app/com.example.app.Main
```

Trong đó:

- -d out: đặt file .class vào thư mục out.
- --module-source-path src: chỉ định thư mục chứa source module.
- -m com.example.app: biên dịch module com.example.app.
- --module-path out: chỉ định nơi tìm module đã biên dịch.
- com.example.app/com.example.app.Main: tên module và class main.

5. requires

requires khai báo module hiện tại phụ thuộc vào module khác.

```
module com.example.app {  
    requires java.sql;  
}
```

Ví dụ dùng JDBC:

```
package com.example.app;  
  
import java.sql.Connection;  
import java.sql.DriverManager;  
import java.sql.SQLException;  
  
public class DatabaseConnection {  
    public static Connection connect() throws SQLException {  
        return DriverManager.getConnection(  
            "jdbc:mysql://localhost:3306/test",  
            "root",  
            "password"  
        );  
    }  
}
```

```
}  
}
```

Module cần có:

```
module com.example.app {  
    requires java.sql;  
}
```

Nếu thiếu `requires java.sql`, compiler có thể báo lỗi vì module không đọc được package `java.sql`.

6. exports

`exports` cho phép module khác truy cập các public type trong package.

```
module com.example.model {  
    exports com.example.model.api;  
}
```

Giả sử package có class:

```
package com.example.model.api;  
  
public class User {  
    private final String name;  
  
    public User(String name) {  
        this.name = name;  
    }  
  
    public String getName() {  
        return name;  
    }  
}
```

Module khác có thể import class này:

```
import com.example.model.api.User;
```

Chỉ package được `exports` mới được public ra ngoài module.

7. Package không được exports

Package không được exports vẫn có thể chứa public class, nhưng module khác không được truy cập trực tiếp.

```
module com.example.model {  
    exports com.example.model.api;  
}
```

Trong ví dụ trên:

```
com.example.model.api      → module khác được phép truy cập  
com.example.model.internal → chỉ dùng bên trong module
```

Đây là điểm khác biệt quan trọng giữa public class và package được exports.

8. requires transitive

requires transitive truyền dependency sang module phụ thuộc.

Module A:

```
module com.example.api {  
    requires transitive com.example.model;  
    exports com.example.api;  
}
```

Module B sử dụng module A:

```
module com.example.app {  
    requires com.example.api;  
}
```

Nhờ requires transitive, module B cũng có thể đọc module com.example.model mà không cần khai báo trực tiếp requires com.example.model.

Chỉ nên dùng requires transitive khi dependency đó thực sự xuất hiện trong API public của module.

9. requires static

requires static khai báo dependency chỉ cần khi biên dịch, không bắt buộc lúc chạy.

```
module com.example.app {  
    requires static lombok;  
}
```

Điều này thường dùng với:

- Annotation processor.
- Thư viện chỉ cần lúc compile.
- Dependency tùy chọn.

Nếu code thực sự cần class của module đó lúc runtime, không nên dùng `requires static`.

10. opens

`exports` dùng cho việc truy cập trực tiếp các public type.

`opens` dùng để cho phép Reflection truy cập các thành phần bên trong package, kể cả private field hoặc private method trong một số trường hợp.

```
module com.example.model {  
    opens com.example.model.entity;  
}
```

Ví dụ framework có thể cần Reflection để đọc field:

```
class User {  
    private String name;  
}
```

Có thể mở package cho riêng một module:

```
module com.example.model {  
    opens com.example.model.entity to com.example.framework;  
}
```

11. open module

Có thể mở toàn bộ package trong module cho Reflection:

```
open module com.example.model {  
    exports com.example.model.api;  
}
```

open module không đồng nghĩa với exports.

- exports: cho phép module khác truy cập trực tiếp package.
- opens: cho phép Reflection truy cập package.
- open module: mở tất cả package cho Reflection.

Không nên dùng open module nếu chỉ cần mở một vài package.

12. exports và opens khác nhau

Khai báo	Mục đích
exports	Cho phép truy cập trực tiếp các public type
opens	Cho phép Reflection truy cập
open module	Mở toàn bộ package cho Reflection
exports ... to	Chỉ export cho module cụ thể
opens ... to	Chỉ mở cho module cụ thể

Ví dụ:

```
module com.example.model {  
    exports com.example.model.api;  
    opens com.example.model.entity to com.example.orm;  
}
```

13. Module phụ thuộc lẫn nhau

Ví dụ có hai module:

```
com.example.model  
com.example.app
```

Module model:

```
module com.example.model {  
    exports com.example.model.api;  
}
```

Module app:

```
module com.example.app {  
    requires com.example.model;
```

```
}
```

Class trong app:

```
package com.example.app;

import com.example.model.api.User;

public class Main {
    public static void main(String[] args) {
        User user = new User("An");
        System.out.println(user.getName());
    }
}
```

14. Module Graph

Các module tạo thành một đồ thị dependency.

```
com.example.app
    |
    | requires
    v
com.example.service
    |
    | requires
    v
com.example.model
```

Module system kiểm tra:

- Module có tồn tại không.
- Dependency có bị thiếu không.
- Có module bị trùng tên không.
- Package có bị split giữa nhiều module không.
- Module có quyền đọc module khác không.

15. Named Module và Unnamed Module

Named Module

Module có file module-info.java.

```
module com.example.app {
}
```

Đây là named module.

Unnamed Module

Code chạy trên classpath, không có module-info.java, thuộc về unnamed module.

```
java -cp out com.example.Main
```

Unnamed module:

- Có thể đọc các module trong hệ thống.
- Không khai báo dependency bằng requires.
- Thường được dùng bởi các project cũ chưa chuyển sang module.

16. Automatic Module

Một JAR không có module-info.java nhưng được đặt trên module path có thể trở thành automatic module.

Tên module có thể được suy ra từ tên file JAR hoặc được khai báo trong MANIFEST.MF.

Ví dụ:

```
library-utils-1.0.jar
```

Có thể được suy ra thành tên module gần giống:

```
library.utils
```

Automatic module:

- Có tên module.
- Có thể được module khác requires.
- Thường exports tất cả package.
- Là bước chuyển tiếp khi thư viện chưa hỗ trợ JPMS đầy đủ.

17. Module của Java Platform

JDK hiện đại cũng được chia thành nhiều module.

Một số module phổ biến:

Module	Nội dung
java.base	Class cơ bản, tự động được yêu cầu
java.sql	JDBC và SQL API

Module	Nội dung
java.desktop	AWT, Swing
java.net.http	HTTP Client
java.logging	Logging API
java.xml	XML API
jdk.httpserver	HTTP Server đơn giản

Module java.base luôn được tự động requires, không cần khai báo:

```
module com.example.app {  
    // Không cần viết requires java.base;  
}
```

18. Service Provider trong Module

Module system hỗ trợ mô hình service provider.

Có ba thành phần:

- Service interface.
- Service provider.
- Service consumer.

Service interface

```
package com.example.service.api;  
  
public interface GreetingService {  
    String greet(String name);  
}
```

Module cung cấp service

```
module com.example.greeting.provider {  
    requires com.example.service.api;  
    provides com.example.service.api.GreetingService  
        with com.example.greeting.provider.VietnameseGreetingService;  
}
```

Class provider:

```
package com.example.greeting.provider;

import com.example.service.api.GreetingService;

public class VietnameseGreetingService implements GreetingService {
    @Override
    public String greet(String name) {
        return "Xin chào " + name;
    }
}
```

Module sử dụng service

```
module com.example.app {
    requires com.example.service.api;
    uses com.example.service.api.GreetingService;
}
```

Đọc service bằng ServiceLoader:

```
import java.util.ServiceLoader;
import com.example.service.api.GreetingService;

ServiceLoader<GreetingService> services =
    ServiceLoader.load(GreetingService.class);

for (GreetingService service : services) {
    System.out.println(service.greet("An"));
}
```

19. module-info.java và main class

module-info.java chỉ khai báo cấu hình module, không chứa class thông thường.

```
module com.example.app {
    exports com.example.app.api;
}
```

Main class vẫn nằm trong package riêng:

```
package com.example.app;

public class Main {
```

```
public static void main(String[] args) {  
    System.out.println("Application started");  
}  
}
```

20. Kiểm tra module bằng công cụ JDK

Xem module của JDK:

```
java --list-modules
```

Xem module descriptor của file JAR:

```
jar --describe-module --file library.jar
```

Xem dependency của module:

```
jdeps --module-path out -s out/com.example.app
```

21. Điểm chính

- Module là nhóm package có tên và descriptor riêng.
- module-info.java là file khai báo module.
- requires khai báo dependency.
- exports công khai package cho module khác.
- opens cho phép Reflection truy cập package.
- uses và provides hỗ trợ ServiceLoader.
- Module path dùng để tìm module.
- Classpath và module path là hai cơ chế khác nhau.
- java.base được tự động yêu cầu.
- Module system kiểm tra dependency và quyền truy cập rõ ràng hơn classpath.

22. Thực hành tốt

- Đặt tên module ổn định, thường dùng dạng tên package đảo ngược.
- Chỉ exports các package thật sự cần public.
- Giữ package implementation ở dạng internal, không exports.
- Chỉ dùng opens cho package cần Reflection.
- Không dùng open module nếu chỉ cần mở một package.
- Khai báo dependency trực tiếp thay vì phụ thuộc ngầm.
- Dùng requires transitive khi dependency xuất hiện trong API public.
- Kiểm tra module path và classpath khi chạy ứng dụng.
- Di chuyển từng phần của project cũ sang module để dễ kiểm soát.

23. Các lỗi thường gặp

Lỗi	Nguyên nhân
package is not visible	Package chưa được module khác exports hoặc module chưa requires
module not found	Thiếu module trên module path
package exists in another module	Split package giữa nhiều module
class not found khi chạy	Sai module path hoặc sai tên main class
Reflection access lỗi	Chưa opens package cho module cần Reflection
package is declared in module but module does not read it	Thiếu requires
Tên module không hợp lệ	Tên chứa ký tự không phù hợp hoặc trùng module khác

24. Ghi nhớ

```

module-info.java = mô tả module
requires          = module này cần module nào?
exports          = công khai package nào?
opens            = cho phép Reflection vào package nào?
uses             = sử dụng service nào?
provides         = cung cấp implementation nào?

```

Ví dụ đầy đủ tối thiểu:

```

module com.example.app {
    requires java.sql;
    exports com.example.app.api;
}

```

Cấu trúc thư mục:

```

src
├── com.example.app
│   ├── module-info.java
│   └── com
│       ├── example
│       │   └── app
│       │       ├── Main.java
│       │       └── api
│       │           └── User.java

```